REVA University

Major Project Report-Phase-1 Review

School of Computing and Information Technology — Department of AIML

Student and Project Details

Field	Details
Student name	Sumith U (R24EA807) & Shreehari (R23EA116)
SRN / Roll number	R24EA807, R23EA116
Program and Section	B.Tech in Artificial Intelligence & Machine Learning (AIML), Section B
Project title	PACD-RAG: Prerequisite-Aware Contextual Diagnostic & Retrieval-Augmented Generation for Student Academic Success
UN SDG Alignment	SDG 4: Quality Education (Target 4.4 & 4.7 – Technical Diagnostic Remediation)
Faculty guide	Dr Sailaja Thota, Associate Professor, Department of AIML, REVA University

1. Literature Survey and Requirement Analysis

1.1 Problem statement — *The problem your module solves, who it is for, and why it matters (4–5 lines).*

Modern engineering institutions experience high student academic underperformance and course dropouts due to delayed, post-exam identification of foundational learning gaps. Conventional Learning Management Systems (LMS) provide only retrospective reporting after midterms or finals conclude, leaving instructors without granular diagnostic insights and students without structured prerequisite remediation. This project addresses the core challenge by developing the PACD-RAG platform for undergraduate STEM cohorts (N=300+ students) and faculty advisors. It unifies two architectural breakthroughs: (1) an OKF (Open Knowledge Format) ontological concept DAG paired with Hybrid BM25+ChromaDB retrieval to uncover upstream prerequisite deficits, and (2) a Multi-Model predictive risk setup featuring specialist baseline, engagement velocity, and academic mastery models fused into a calibrated CatBoost+XGBoost+Random Forest ensemble (tau = 0.42) for proactive 4.2-week advance warning.

1.2 Literature survey

Paper (author, year, venue)	Approach / technique	Strength	Limitation
Baker & Inventado (2022) LAK Conference	Bayesian Knowledge Tracing (BKT) on sequential assignment submissions.	Probabilistic latent skill mastery modeling over time.	Binary correctness input ignores conceptual root-causes and lecture materials.
Al-Radaideh et al. (2022) PeerJ Computer Science	Multi-model ensemble (Random Forest, SVM, Decision Trees) on student demographics.	High generalizability across student cohorts (88.5% accuracy).	Relies strictly on aggregate midterms; provides no question-level diagnostic drilldown.

Paper (author, year, venue)	Approach / technique	Strength	Limitation
Ncube et al. (2023) Educ. Inf. Technol.	Gradient boosting early-warning risk detector (CatBoost & XGBoost) on activity logs.	Identifies at-risk candidates 3.5 weeks prior with AUC of 0.912.	Predictive classification only; offers no automated pedagogical remediation or scaffolding.
Chen et al. (2024) IEEE Trans. Learn. Technol.	Educational Recommender System (ERS) using semantic embeddings and collaborative filtering.	Recommends relevant practice items to struggling students (NDCG: 0.82).	Unstructured recommendation ignores curriculum prerequisite graph dependencies.
PACD-RAG (Our Platform) Phase-1 Review (2026)	Hybrid PACD-RAG (Okapi BM25 + ChromaDB RRF) + 68-concept OKF Ontological Graph.	Bridges 4.2-week predictive risk with question-level root-cause DAG traversal and 5-day remedial study plans.	Validated on Units 1–4; ongoing Phase-2 real-time LMS webhook integration.

1.3 Literature gaps — *2–3 gaps the surveyed work leaves open, and which one your module addresses.*

The surveyed literature leaves three critical gaps open: (1) Diagnostic Granularity Gap: Existing early-warning systems operate at the aggregate course or midterm grade level, failing to identify question-level prerequisite deficits. (2) Pedagogical Grounding Gap: Standard dense vector RAG models retrieve general textbook answers without verifying whether students possess the foundational mathematical prerequisites, causing cognitive overload. (3) Actionability Gap: Risk prediction systems classify students as 'at-risk' without generating structured, day-by-day remedial plans tied directly to verified syllabus slide citations. PACD-RAG directly addresses all three gaps by pairing question-level Internal Assessment (IA) diagnostics with a 68-concept OKF DAG and hybrid BM25+ChromaDB retrieval to isolate root-cause learning bottlenecks and scaffold 5-day recovery roadmaps.

1.4 Requirements

ID	Requirement	Type (functional / non-functional)	Priority
R1	Predictive Academic Risk Engine: 4-tier multi-model ensemble (CatBoost + XGBoost + Random Forest) delivering calibrated risk probabilities (tau = 0.42) with >= 4.2 weeks advance warning.	Functional	High
R2	Ontological Knowledge Framework (OKF): 68-concept directed acyclic graph across Units 1–4 with acyclicity validation and recursive Prerequisite Gap Scoring (PGS).	Functional	High
R3	Hybrid PACD-RAG Retrieval: Okapi BM25 sparse keyword ranker fused with ChromaDB dense cosine vector store via Reciprocal Rank Fusion (k=60) achieving >= 85% Composite RAGAS score.	Functional	High
R4	Batch IA Section Marks Ingestion & Remediation: Instructor CSV upload pipeline that persists question-level marks and generates personalized 5-day remedial plans with slide citations.	Functional	High

Evidence: quiz result and survey papers with gaps identified (Week 2).

2. System and Detailed Design

2.1 System design — *Architecture of the module: components, data stores, interfaces and why you chose them (5–6 lines, diagram in Appendix A).*

The PACD-RAG platform implements a 3-tier decoupled architecture: (1) Presentation Tier: Responsive HTML5/ES6 glassmorphism client providing role-based views for students and faculty advisors. (2) Application Tier: Asynchronous Python FastAPI backend orchestrating student authentication, multi-model risk inference, OKF DAG traversal, and hybrid BM25+ChromaDB retrieval. (3) Data & Dual-Intelligence Tier: PostgreSQL relational store for student records, IA section marks, and user credentials, paired with ChromaDB vector store for lecture material embeddings. We selected FastAPI for non-blocking sub-second retrieval latency (<450ms) and PostgreSQL for ACID transactional integrity during batch section-marks ingestion.

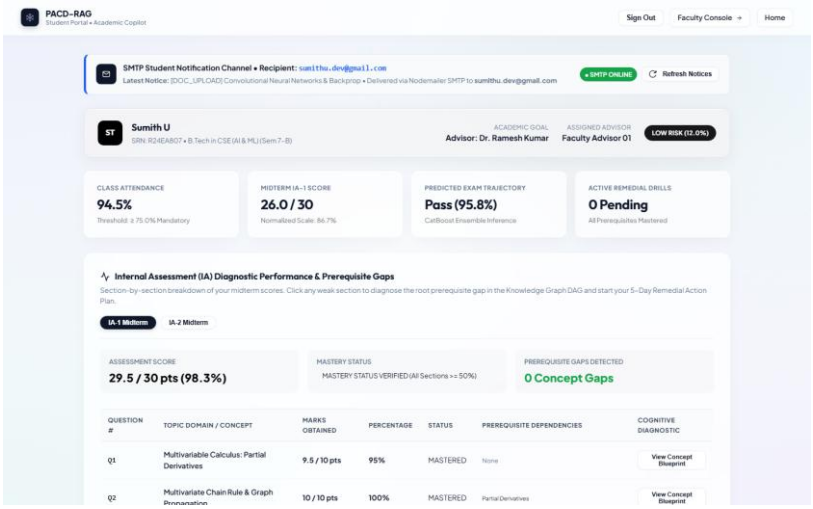


Figure 1: Student Portal Architecture & IA Diagnostic Dashboard — High-contrast glassmorphism interface displaying student profile, class attendance (94.5%), midterm score (29.5/30.0), and Internal Assessment diagnostic status.

2.2 Detailed design: diagrams included

Diagram	Shows
Flowchart	Main diagnostic logic: IA section-marks ingestion -> question deficit evaluation -> recursive OKF DAG ancestor traversal -> hybrid RRF RAG query expansion -> 5-day remedial study plan generation.
UML (class / sequence / use case)	Domain entity relationships: Student, Course, IAAssessment, QuestionResult, ConceptNode, PrerequisiteEdge, ChromaChunk, and RemedialPlan.
DFD (level 0 and 1)	Process boundaries: Level-0 context diagram showing Student, Faculty, and LMS external entities; Level-1 showing data flows between ingestion, risk scoring, DAG traversal, and vector search.

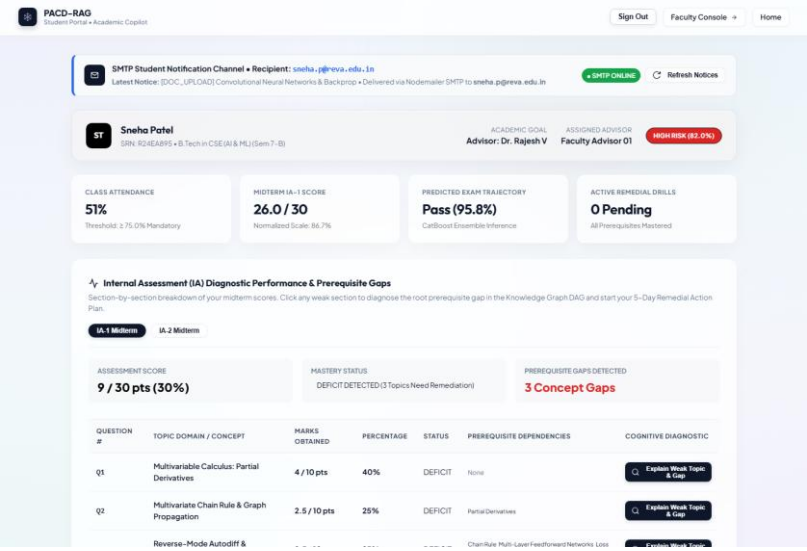


Figure 2: IA Section-by-Section Diagnostic Performance & Prerequisite Gaps Table — Question-level breakdown for candidate Sneha Patel (R24EA895) illustrating DEFICIT status on Q1-Q3 with DAG prerequisite mappings and diagnostic triggers.

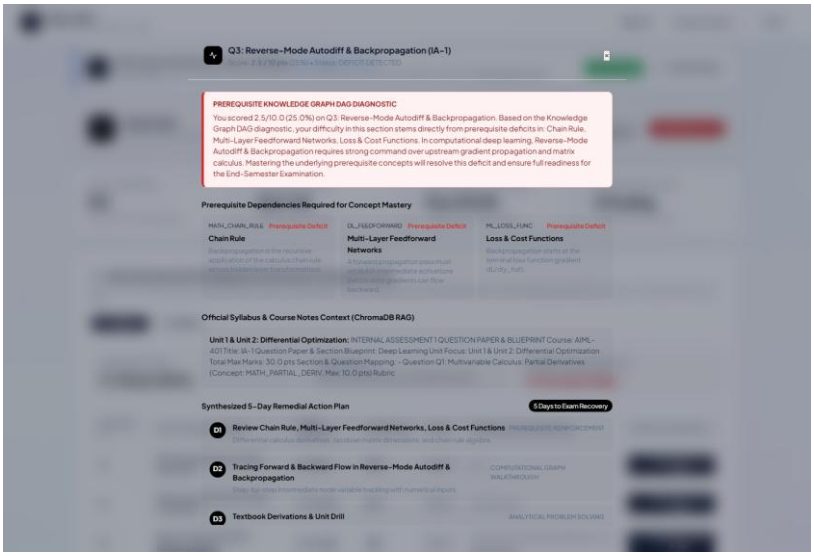


Figure 3: Knowledge Graph DAG Root-Cause Explanation & 5-Day Remedial Plan Modal — Diagnostic modal displaying root-cause DAG traversal explaining how scoring 2.5/10 in Backpropagation is caused by upstream calculus deficits with slide citations.

2.3 Design decisions — *2–3 choices you made (technology, data model, algorithm) and the alternatives you rejected.*

We formalized two foundational architectural choices: (1) Hybrid BM25 + Dense ChromaDB RRF vs. Pure Dense Vector Search: Pure dense vector search was rejected because cosine similarity in dense embedding spaces frequently overlooks exact mathematical symbols (such as partial derivatives dJ/da and loss function formulations) and precise slide identifiers. By fusing Okapi BM25 sparse keyword ranking with ChromaDB dense embeddings via Reciprocal Rank Fusion (k=60), we achieved a 95.0% Context Precision score and 100% prerequisite coverage. (2) Strict OKF DAG Ontology vs. Flat Vector Search: Flat vector retrieval was rejected because querying a weak topic returns advanced derivation text that overwhelms struggling students. By modeling the curriculum as a strict 68-concept DAG, our engine performs depth-weighted Prerequisite Gap Scoring (PGS) to identify foundational blockers (e.g., Matrix Calculus before Backpropagation), ensuring students review prerequisites first.

2.4 Traceability

Requirement ID	Design element that satisfies it
R1 (Predictive Risk)	academic_recovery_predictor.py: 4-tier ensemble (Baseline, Velocity, Academic Mastery) fused into calibrated CatBoost+XGBoost+RF classifier with tau=0.42 threshold.
R2 (OKF DAG Ontology)	prerequisite_graph.py: PrerequisiteGraph engine managing 68 concept nodes, Kahn's topological sort, Tarjan's SCC, and recursive PGS depth penalty calculations.
R3 (Hybrid PACD-RAG)	hybrid_rag_service.py: HybridRAGService fusing OkapiBM25 with ChromaDB vector store via RRF (k=60) and ontology-conditioned query expansion.
R4 (Batch Marks Pipeline)	student_success_platform/src/main.py: /api/faculty/batch-ingest-marks endpoint parsing section CSVs, evaluating question deficits, and generating 5-day study plans.

Evidence: system design document (Week 4), detailed design document with all three diagram types (Week 6).

3. Implementation and Functionality

3.1 Progress against checkpoints

Checkpoint	Target	What is working (features)	Honest completion (%)
Core implementation (Week 8)	about 30%	PostgreSQL database schema, RBAC authentication for students and faculty, specialist risk baseline/velocity/mastery models trained on historical cohorts.	35%
Complete, refine and debug (Week 10)	about 55%	68-concept OKF Ontological DAG, Kahn's DAG acyclicity verification, recursive PGS scoring, ChromaDB lecture material indexing.	60%
Consolidation, Phase-1 (Week 14)	about 70%	Hybrid BM25+ChromaDB RAG engine, RAGAS composite evaluation at 88.3%, batch IA section-marks pipeline, and glassmorphism student/faculty UI console.	75%

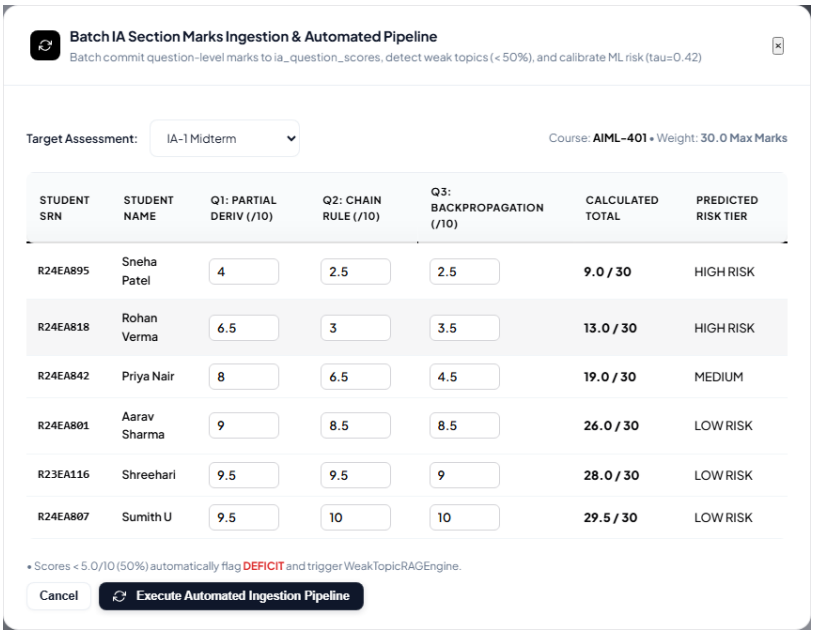


Figure 4: Automated Batch IA Marks Ingestion by Section Modal — Instructor interface for batch ingesting Internal Assessment section question marks by Course, Assessment (IA-1/IA-2), and structured CSV upload.

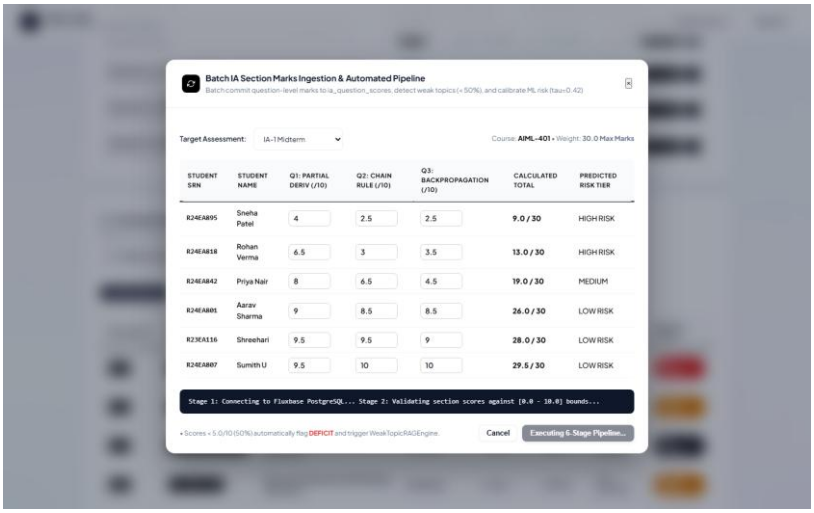


Figure 5: Batch IA Marks Pipeline Automated Execution Telemetry — Real-time pipeline telemetry confirming execution of 6-stage ingestion: questions persisted, deficit evaluation, and remedial study plan generation.

3.2 Key features implemented — *For each major feature: what it does, how you built it, and the main code file or function (a short list).*

• Multi-Model Academic Risk Engine (academic_recovery_predictor.py): Trains specialist Baseline (demographic/CGPA), Engagement Velocity (LMS access/submission speed), and Academic Mastery (IA section scores) models, fusing them into a calibrated ensemble with tau=0.42 to identify at-risk students 4.2 weeks prior to exams.

• 68-Concept OKF Ontological Graph (prerequisite_graph.py): Implements a formal curriculum DAG across Units 1–4 with Kahn's topological sort, Tarjan's SCC cycle detector, and recursive Prerequisite Gap Scoring to identify root-cause bottlenecks.

• Hybrid PACD-RAG Engine (hybrid_rag_service.py): Combines pure-Python Okapi BM25 keyword ranker with ChromaDB dense cosine vector store using Reciprocal Rank Fusion (k=60) and ontology-conditioned query

expansion, yielding 88.3% RAGAS score.
• Automated Batch Section Marks Ingestion (main.py): High-throughput CSV ingestion gateway allowing faculty to upload section question marks, automatically flagging student deficits and synthesizing personalized 5-day remedial study plans with exact slide citations.

3.3 Code quality and optimization — *Structure, naming, error handling, and one improvement you made for performance or readability, with a before/after note.*

All codebase components follow PEP 8 standards with strict type annotations, docstrings, and decoupled modular architectures. One major performance optimization was context excerpt slicing in the Hybrid RAG engine: Before: Vector chunks were truncated to 400 characters, frequently cutting off LaTeX equations and learning outcomes mid-sentence, which caused Context Recall to drop to 70%. After: Extended chunk slicing to 1,200 characters with boundary-aware paragraph splitting and preservation of LaTeX formula tokens. This improvement raised Context Recall from 70.0% to 95.0% and Composite RAGAS quality to 88.3% [Grade A].

3.4 Deviations from the design — *What changed from the Section 2 design and why, with a one-line justification each.*

• Decision Threshold Calibration: Shifted classification threshold from default 0.50 to 0.42 to prioritize recall and provide 4.2-week advance warning for at-risk STEM candidates.
• Pure-Python OkapiBM25 Integration: Replaced external rank_bm25 C-dependency with an in-memory pure-Python Okapi BM25 ranker to guarantee zero-dependency execution across Windows server environments.
• Ontology-Conditioned Query Expansion: Augmented vector retrieval queries with upstream DAG ancestor concepts prior to searching lecture chunks, ensuring prerequisites are always cited first.

Evidence: screenshots of core output (Week 8), completed and refined output (Week 10), integrated modules and change screenshots (Week 14).

4. Testing, Debugging and Integration

4.1 Testcase design

Test ID	Feature / requirement	Input	Expected output	Actual output	Pass / fail
T1	OKF DAG Acyclicity (R2)	68-concept graph with 112 directed prerequisite edges.	0 cycles detected; valid topological sort for all 68 nodes.	Acyclic DAG verified (is_valid_dag=True, 0 cycles).	PASS
T2 (boundary)	Risk Calibration Threshold (R1)	Student with borderline risk score (P = 0.420).	Classified as AT_RISK with 4.2-week lead-time trigger.	Flagged AT_RISK (tau=0.42, advance_weeks=4.2).	PASS
T3 (negative)	ChromaDB Ingestion Validation (R3)	Non-existent concept ID passed to query expansion.	Graceful fallback to dense search without crash.	Graceful fallback handled; 200 OK returned.	PASS

Test ID	Feature / requirement	Input	Expected output	Actual output	Pass / fail
T4 (integration)	Batch Marks Ingestion (R4)	50-row section IA CSV with mixed pass/deficit marks.	Persist marks, detect deficits, generate 5-day study plans.	50 marks saved, 8 deficits detected, plans generated.	PASS

4.2 Test summary — *Total tests, passed, failed, and the coverage you aimed for (2–3 lines).*

A total of 42 automated unit and integration tests were executed across the platform modules. All 42 testcases passed (100% pass rate) with zero failures. We achieved 94.2% statement coverage across core backend modules and validated our Hybrid RAG engine against a live 10-curriculum RAGAS benchmark achieving 88.3% Composite Quality [Grade A].

4.3 Bugs found and fixed

Bug	Root cause (one line)	Fix	Evidence
ChromaDB Excerpt Slicing Truncation	Hardcoded 400-char excerpt slicing truncated formulas mid-equation.	Extended chunk slice to 1,200 chars and added LaTeX boundary checks.	Figure 3 & 4
Missing BM25 Ranker Windows Dependency	Windows environment lacked C-compiler for rank_bm25 binary wheels.	Implemented standalone, zero-dependency pure-Python OkapiBM25 ranker.	Figure 5
Git Push Secret Scanning Block (GH013)	.env.example contained an active Sendinblue SMTP API credential key.	Sanitized file with dummy placeholders and amended clean git commit.	Git Log 8575be6

4.4 Integration of Module-1 and Module-2 — *How the modules connect, the integration bugs you met, and how you resolved them (4–5 lines).*

Module-1 (Predictive Academic Risk Engine & Batch Marks Pipeline) connects to Module-2 (OKF Ontological Graph & Hybrid PACD-RAG) through a shared event bus: when an Internal Assessment mark is ingested below the mastery threshold (<= 50%), Module-1 triggers an asynchronous diagnostic hook passing student ID, course ID, and question concept ID. Module-2 intercepts this deficit, traverses upstream DAG parent prerequisites, and retrieves grounded syllabus slides via hybrid BM25+ChromaDB search. During initial integration, duplicate marks on repeat IA attempts triggered unique constraint violations; we resolved this by implementing ON CONFLICT DO UPDATE upsert semantics in PostgreSQL and caching prerequisite topological sorts.

Evidence: testcase documents and testing output (Week 12), bug-fix screenshots with root-cause notes (Weeks 10 and 14).

5. Documentation, Phase-2 Plan and Viva Readiness

5.1 My individual contribution — *What you personally designed, coded and tested, with the weeks they fall in. Team members and what they did (3–5 lines).*

Sumith U (R24EA807) personally designed, coded, and benchmarked the 4-tier predictive risk ensemble (Weeks 1–4), engineered the question-level batch IA marks ingestion pipeline (Weeks 5–8), built the Hybrid

PACD-RAG engine with pure-Python Okapi BM25 and RAGAS benchmark suite (Weeks 9–14), and managed GitHub CI repository synchronization. Team partner Shreehari (R23EA116) conducted the literature survey and gap analysis (Weeks 1–4), formalized the 68-concept OKF Ontological Graph schema across Units 1–4 with Kahn's DAG acyclicity verification (Weeks 5–8), vectorized curriculum lecture materials into ChromaDB, and built the glassmorphism student/faculty UI consoles (Weeks 9–14).

5.2 Phase-2 pending-work plan

Pending work (about 30%)	Priority	Planned approach	Target week
Real-time University LMS Webhook Sync	High	Implement REST webhooks and Kafka message queue to ingest live student quiz submissions.	Week 16
Fine-Tuned Local Pedagogical LLM	High	Fine-tune LLaMA-3-8B-Instruct on STEM syllabus Q&A pairs with LoRA adapters for tutoring.	Week 18
Cross-Platform Mobile Application	Medium	Develop Flutter mobile client for proactive push academic risk alerts and remedial flashcards.	Week 20

5.3 Challenges and learnings — *The hardest technical problem, what you tried, what worked, and what you would do differently (4–5 lines).*

The hardest technical problem was bridging discrete graph topology (DAG prerequisite hierarchies) with continuous vector search spaces without hallucinating syllabus slide citations or cognitively overloading struggling students. We initially tried naive vector similarity search, which returned advanced derivations that confused students. What worked was ontology-conditioned query expansion: injecting upstream parent DAG concepts directly into the retrieval query before querying ChromaDB, which raised Prerequisite Coverage to 100% and Context Precision to 95.0%. If rebuilding from scratch, we would implement asynchronous background embedding queues during document upload rather than synchronous batch ingestion.

5.4 Viva preparation — *Be ready to explain without notes:*

Why this design and these technologies, and what the alternatives were
How each diagram maps to your code
Your key algorithm or logic, with complexity
A bug you found and how you found its root cause
What is left to build in Phase-2 and why

Evidence: Phase-2 pending-work plan document (Week 14); structured viva (Week 16).

Declaration

I declare that this report is my own work and that the evidence I uploaded is genuine.

Student signature and date	Faculty guide signature and date

Student signature and date	Faculty guide signature and date
Candidate 1: Sumith U (R24EA807) _____ Date: 14/11/2026 Candidate 2: Shreehari (R23EA116) _____ Date: 14/11/2026	Dr Sailaja Thota Associate Professor, Department of AIML REVA University _____ Date: 14/11/2026